

Writing Image Benchmark System 사용 설명서

작성일: 2026-05-03

프로젝트 위치: C:\Project\writing_project

1. 프로그램 소개

이 프로그램은 학생의 영어 작문 이미지와 정답 분석지 JSON을 입력받아, 로컬 Ollama 비전 언어 모델이 얼마나 정확하게 작문을 분석하는지 비교하는 벤치마크 시스템입니다.

현재 비교 대상 모델은 다음과 같습니다.

- `qwen2.5vl:latest`: 큰 비전 언어 모델입니다. 응답이 느릴 수 있지만 이미지 이해 성능을 비교하기 위해 사용합니다.
- `gemma4:latest`: 현재 API 후보 모델입니다. 상대적으로 빠르게 응답하는지 확인합니다.

프로그램은 모델별로 다음 항목을 분석합니다.

- 이미지에서 학생 작문을 읽었는지
- 정해진 JSON 구조로 답변했는지
- 정답지와 비교했을 때 원문, 교정문, 점수, 오류 태그가 얼마나 일치하는지
- 응답 속도와 API 사용 적합성이 어느 정도인지
- 결과 리포트가 HTML, JSON, Markdown 형태로 저장되는지

2. 전체 동작 흐름

1. 사용자가 웹 화면에서 작문 이미지, 프롬프트, 정답 JSON을 입력합니다.
2. FastAPI 서버가 입력 파일을 임시 폴더에 저장합니다.
3. `benchmark_runner.py`가 모델 목록을 읽고 모델별 분석을 실행합니다.
4. `ollama_client.py`가 Ollama의 `/api/generate` 엔드포인트로 이미지와 프롬프트를 보냅니다.
5. 모델 응답을 `output_parser.py`가 JSON으로 파싱합니다.
6. `comparator.py`와 `diff_engine.py`가 정답지와 모델 출력값을 비교합니다.
7. 결과가 `results` 폴더 아래에 저장됩니다.
8. 웹 화면과 HTML 리포트에서 모델별 비교 결과를 확인합니다.

3. 실행 전 준비 사항

Python 3.10 이상이 필요합니다. 현재 점검 기준 Python 3.11.9에서 정상 동작했습니다.

필수 패키지 설치:

```
cd C:\Project\writing_project
pip install -r requirements.txt
```

Ollama 실행 확인:

```
ollama list
```

필요 모델:

```
ollama pull qwen2.5vl:latest
ollama pull gemma4:latest
```

헬스 체크:

```
python -m src.main --health-check
```

정상이라면 PASS가 출력되고, 로그는 results\logs\health_check_YYYYMMDD_HHMMSS.json에 저장됩니다.

4. 웹 서버 기동 방법

PowerShell에서 다음 명령을 실행합니다.

```
cd C:\Project\writing_project
uvicorn src.api:app --reload --host 127.0.0.1 --port 8000
```

접속 URL:

```
http://127.0.0.1:8000/
```

확인용 URL:

```
http://127.0.0.1:8000/health
http://127.0.0.1:8000/models
```

5. 웹 화면 사용법

웹 화면에서 다음 순서로 실행합니다.

1. 학생 작문 이미지 파일을 선택합니다.
2. 프롬프트 영역을 확인합니다. 기본값은 data\prompts\system_prompt.txt와 data\prompts\output_prompt.txt를 합친 내용입니다.
3. 정답 분석지 JSON을 입력합니다. data\expected_outputs 폴더의 최신 JSON을 불러올 수도 있습니다.
4. 필요하다면 실행 이름을 입력합니다. 비워두면 api_YYYYMMDD_HHMMSS 형태로 자동 생성됩니다.
5. 벤치마크 실행 버튼을 누릅니다.
6. 진행률과 현재 모델 실행 상태를 확인합니다.
7. 완료 후 화면 하단의 모델별 결과와 비교표를 확인합니다.

qwen 모델은 첫 응답까지 시간이 오래 걸릴 수 있습니다. 이 경우 화면에 모델 응답 대기 중과 경과 시간이 표시됩니다. 멈춘 것이 아니라 모델이 이미지를 처리하고 있는 상태입니다.

6. 입력 데이터 구성

이미지 파일

지원 확장자:

- .jpg
- .jpeg
- .png
- .bmp
- .gif
- .webp

이미지는 웹 화면에서 직접 업로드합니다. 샘플을 폴더에 보관하려면 `dataWinput_images`를 사용하면 됩니다.

정답 분석지 JSON

정답지는 `dataWexpected_outputs` 폴더에 둘 수 있습니다. 웹 화면에서는 정답 JSON을 직접 붙여넣거나 서버에 있는 최신 파일을 불러올 수 있습니다.

주요 구조:

```
{
  "metadata": {},
  "original_writing": "",
  "corrected_writing": "",
  "scoring_analysis": {},
  "writing_performance": {},
  "error_explanations": [],
  "error_analysis": {},
  "overall_comments": ""
}
```

정답지는 모델 평가 기준이므로 가능한 한 사람이 검수한 정확한 JSON을 넣어야 합니다.

7. 분석 방법

프로그램은 모델별 출력값을 정답지와 비교합니다.

JSON 파싱 검사

모델이 순수 JSON을 출력했는지 확인합니다. 코드 블록이나 설명 문장이 섞여 있어도 가능한 경우 JSON 객체를 추출하려고 시도합니다.

관련 소스:

- `srcWoutput_parser.py`

스키마 준수 검사

필수 top-level key가 있는지, 점수 구조가 맞는지, 점수 합계가 유효한지 검사합니다.

관련 소스:

- `srcWjson_validator.py`

정답지 항목별 비교

정답 JSON과 모델 JSON을 leaf 항목 단위로 펼쳐 비교합니다.

예:

```
metadata.class
정답 입력값: B2
모델 출력값: C1
상태: different
```

또는:

```
scoring_analysis.grammar.subtotal.score
정답 입력값: 26
모델 출력값: 24
상태: off_by_small
```

관련 소스:

- `srcWdiff_engine.py`
- `srcWhtml_reporter.py`
- `srcWapi.py`

원문과 교정문 유사도

`original_writing`과 `corrected_writing`을 정답지와 비교합니다. 오류 태그는 제거한 뒤 텍스트 유사도를 계산합니다.

관련 소스:

- `srcWmetrics.py`

오류 태그 비교

오류 태그는 다음 의미입니다.

- `[G]`: Grammar
- `[V]`: Vocabulary
- `[O]`: Word Order
- `[P]`: Punctuation
- `[S]`: Spelling

- [C]: Coherence

모델이 `original_writing`에 태그를 넣지 않았더라도, `error_explanations`나 `error_analysis`에 있는 정보를 보조로 사용해 비교합니다.

관련 소스:

- `src\metrics.py`
- `src\diff_engine.py`

점수 비교

`scoring_analysis`의 세부 항목과 `writing_performance`의 점수를 비교합니다.

비교 상태 예:

- `match`: 완전 일치
- `near_match`: 텍스트가 거의 일치
- `off_by_small`: 숫자 차이가 작음
- `off_by_large`: 숫자 차이가 큼
- `missing`: 모델 출력에 항목 없음
- `extra`: 모델 출력에만 있는 항목
- `different`: 값이 다름

8. 결과 확인 방법

실행이 끝나면 결과는 `results` 폴더 아래에 저장됩니다.

주요 저장 위치:

```
results\raw_outputs\<run_name>\<model>_raw.txt
results\parsed_outputs\<run_name>\<model>_parsed.json
results\comparisons\<run_name>\<model>_comparison.json
results\reports\<run_name>\benchmark_summary.json
results\reports\<run_name>\benchmark_report.md
results\reports\<run_name>\benchmark_report.html
results\logs\<run_name>\run.log
```

가장 보기 쉬운 파일은 다음 HTML 리포트입니다.

```
results\reports\<run_name>\benchmark_report.html
```

웹 화면에서는 실행 완료 후 바로 요약 결과와 모델별 상세 비교를 볼 수 있습니다.

9. 프로그램 폴더 구성

```
C:\Project\writing_project
|— config
|   |— benchmark_config.json
```

```
|   └─ models.json
└─ data
    │   └─ expected_outputs
    │   └─ input_images
    │   └─ prompts
└─ docs
└─ results
    │   └─ comparisons
    │   └─ logs
    │   └─ parsed_outputs
    │   └─ raw_outputs
    │   └─ reports
└─ src
└─ tests
```

10. 주요 소스 위치

웹 서버와 화면

- srcWapi.py

FastAPI 앱과 웹 대시보드 HTML/JavaScript가 들어 있습니다. `/`, `/health`, `/models`, `/api/run`, `/api/run/{job_id}` 엔드 포인트를 제공합니다.

벤치마크 실행 흐름

- srcWbenchmark_runner.py

이미지 로딩, 모델 호출, 파싱, 비교, 리포트 저장의 전체 순서를 담당합니다.

Ollama 호출

- srcWollama_client.py

Ollama 서버 연결 확인, 모델 목록 조회, 이미지 기반 생성 요청을 담당합니다.

프롬프트 생성

- srcWprompt_builder.py

시스템 프롬프트와 출력 프롬프트를 합치고, qwen용 compact prompt를 생성합니다.

JSON 파싱

- srcWoutput_parser.py

모델 응답에서 JSON 객체를 추출하고 파싱합니다.

검증과 비교

- srcWjson_validator.py

- src\comparator.py
- src\metrics.py
- src\diff_engine.py

스키마 준수, 점수 수학, 텍스트 유사도, 오류 태그, 항목별 차이를 계산합니다.

리포트 생성

- src\report_generator.py
- src\html_reporter.py

JSON 요약, Markdown 리포트, HTML 리포트를 생성합니다.

테스트

- tests

파서, 비교기, 헬스 체크, 모델 클라이언트 mock 테스트가 들어 있습니다.

11. 모델 설정

모델 설정 파일:

config\models.json

현재 설정:

- qwen2.5vl:latest
- timeout: 900초
- retry: 0회
- prompt mode: compact
- format: json
- num_predict: 8192
- gemma4:latest
- timeout: 300초
- retry: 1회
- prompt mode: full
- format: json
- num_predict: 4096

qwen은 응답이 길거나 느릴 수 있어 num_predict를 크게 잡고 JSON 출력 강제를 적용했습니다.

12. CLI 실행 방법

웹 화면 대신 명령줄에서도 실행할 수 있습니다.

```
cd C:\Project\writing_project
python -m src.main `
  --image data/input_images/sample.jpg `
  --system-prompt data/prompts/system_prompt.txt `
  --output-prompt data/prompts/output_prompt.txt `
  --expected data/expected_outputs/sample_expected.json `
  --run-name sample_run_001
```

이미 합쳐진 프롬프트를 사용할 때:

```
python -m src.main `
  --image data/input_images/sample.jpg `
  --prompt data/prompts/combined_eval_prompt.txt `
  --expected data/expected_outputs/sample_expected.json `
  --run-name sample_run_002
```

13. 자주 발생하는 문제

15% 이후 멈춘 것처럼 보임

qwen 모델이 첫 응답을 보내기 전까지 이미지 처리와 추론 준비에 오래 걸릴 수 있습니다. 현재는 **모델 응답 대기** 중과 경과 시간이 표시되도록 수정되어 있습니다.

qwen 결과가 FAIL로 나옴

주요 원인은 JSON이 중간에 잘리거나, 코드 블록 형태로 출력되거나, 필수 항목이 빠지는 경우입니다. 현재 설정은 **format: json**과 **num_predict: 8192**로 보완되어 있습니다.

모델이 보이지 않음

다음 명령으로 설치된 모델을 확인합니다.

```
ollama list
```

없으면 다음 명령으로 받습니다.

```
ollama pull qwen2.5vl:latest
ollama pull gemma4:latest
```

Health Check 실패

Ollama 서버가 실행 중인지 확인합니다.

```
ollama serve
```

다시 헬스 체크를 실행합니다.

```
python -m src.main --health-check
```

14. 유지보수 시 참고 사항

- 모델 목록이나 timeout을 바꾸려면 `config\models.json`을 수정합니다.
- 전체 API 기본 설정은 `config\benchmark_config.json`을 수정합니다.
- 프롬프트 기준을 바꾸려면 `data\prompts\system_prompt.txt`와 `data\prompts\output_prompt.txt`를 수정합니다.
- 결과 비교 방식은 `src\diff_engine.py`와 `src\metrics.py`를 수정합니다.
- 웹 화면 표시 방식은 `src\api.py`를 수정합니다.
- 저장 리포트 HTML은 `src\html_reporter.py`를 수정합니다.

15. 빠른 실행 요약

```
cd C:\Project\writing_project
python -m src.main --health-check
uvicorn src.api:app --reload --host 127.0.0.1 --port 8000
```

브라우저 접속:

```
http://127.0.0.1:8000/
```

결과 리포트 확인:

```
results\reports\<run_name>\benchmark_report.html
```